

如果其他代码也需要访问控制变量，则变量必须定义在语句的外部：

```
// 寻找第一个负值元素
auto beg = v.begin();
while (beg != v.end() && *beg >= 0)
    ++beg;
if (beg == v.end())
    // 此时我们知道 v 中的所有元素都大于等于 0
```

因为控制结构定义的对象的价值马上要由结构本身使用，所以这些变量必须初始化。

5.2 节练习

练习 5.4：说明下列例子的含义，如果存在问题，试着修改它。

```
(a) while (string::iterator iter != s.end()) { /* ... */ }
(b) while (bool status = find(word)) { /* ... */ }
    if (!status) { /* ... */ }
```

5.3 条件语句

C++语言提供了两种按条件执行的语句。一种是 `if` 语句，它根据条件决定控制流；另外一种 `switch` 语句，它计算一个整型表达式的值，然后根据这个值从几条执行路径中选择一条。



5.3.1 `if` 语句

175

if 语句 (`if statement`) 的作用是：判断一个指定的条件是否为真，根据判断结果决定是否执行另外一条语句。`if` 语句包括两种形式：一种含有 `else` 分支，另外一种没有。简单 `if` 语句的语法形式是

```
if (condition)
    statement
```

if else 语句 的形式是

```
if (condition)
    statement
else
    statement2
```

在这两个版本的 `if` 语句中，*condition* 都必须用圆括号包围起来。*condition* 可以是一个表达式，也可以是一个初始化了的变量声明（参见 5.2 节，第 155 页）。不管是表达式还是变量，其类型都必须能转换成（参见 4.11 节，第 141 页）布尔类型。通常情况下，*statement* 和 *statement2* 是块语句。

如果 *condition* 为真，执行 *statement*。当 *statement* 执行完成后，程序继续执行 `if` 语句后面的其他语句。

如果 *condition* 为假，跳过 *statement*。对于简单 `if` 语句来说，程序继续执行 `if` 语句后面的其他语句；对于 `if else` 语句来说，执行 *statement2*。

使用 if else 语句

我们举个例子来说明 if 语句的功能，程序的目的是把数字形式表示的成绩转换成字母形式。假设数字成绩的范围是从 0 到 100（包括 100 在内），其中 100 分对应的字母形式是“A++”，低于 60 分的成绩对应的字母形式是“F”。其他成绩每 10 个划分成一组：60 到 69（包括 69 在内）对应字母“D”、70 到 79 对应字母“C”，以此类推。使用 vector 对象存放字母成绩所有可能的取值：

```
const vector<string> scores = {"F", "D", "C", "B", "A", "A++"};
```

我们使用 if else 语句解决该问题，根据成绩是否合格执行不同的操作：

```
// 如果 grade 小于 60，对应的字母是 F；否则计算其下标
string lettergrade;
if (grade < 60)
    lettergrade = scores[0];
else
    lettergrade = scores[(grade - 50)/10];
```

判断 grade 的值是否小于 60，根据结果选择执行 if 分支还是 else 分支。在 else 分支中，由成绩计算得到一个下标，具体过程是：首先从 grade 中减去 50，然后执行整数除法（参见 4.2 节，在 125 页），去掉余数后所得的商就是数组 scores 对应的下标。

嵌套 if 语句

176

接下来让我们的程序更有趣点儿，试着给那些合格的成绩后面添加一个加号或减号。如果成绩的末位是 8 或者 9，添加一个加号；如果末位是 0、1 或 2，添加一个减号：

```
if (grade % 10 > 7)
    lettergrade += '+'; // 末尾是 8 或者 9 的成绩添加一个加号
else if (grade % 10 < 3)
    lettergrade += '-'; // 末尾是 0、1 或者 2 的成绩添加一个减号
```

我们使用取模运算符（参见 4.2 节，第 125 页）计算余数，根据余数决定添加哪种符号。

接着把这段添加符号的代码整合到转换成成绩形式的代码中去：

```
// 如果成绩不合格，不需要考虑添加加号减号的问题
if (grade < 60)
    lettergrade = scores[0];
else {
    lettergrade = scores[(grade - 50)/10]; // 获得字母形式的成绩
    if (grade != 100) // 只要不是 A++，就考虑添加加号或减号
        if (grade % 10 > 7)
            lettergrade += '+'; // 末尾是 8 或者 9 的成绩添加一个加号
        else if (grade % 10 < 3)
            lettergrade += '-'; // 末尾是 0、1 或者 2 的成绩添加一个减号
}
```

注意，我们使用花括号把第一个 else 后面的两条语句组合成了一个块。如果 grade 不小于 60 要做两件事：从数组 scores 中获取对应的字母成绩，然后根据条件设置加号或减号。

注意使用花括号

有一种常见的错误：本来程序中有几条语句应该作为一个块来执行，但是我们忘了用花括号把这些语句包围。在下面的例子中，添加加号减号的代码将被无条件地执行，这显然违背了我们的初衷：

```
if (grade < 60)
    lettergrade = scores[0];
else // 错误：缺少花括号
    lettergrade = scores[(grade - 50)/10];
// 虽然下面的语句从形式上看有缩进，但是因为没有花括号，
// 所以无论什么情况都会执行接下来的代码
// 不及格的成绩也会添加上加号或减号，这显然是错误的
if (grade != 100)
    if (grade % 10 > 7)
        lettergrade += '+'; // 末尾是 8 或者 9 的成绩添加一个加号
    else if (grade % 10 < 3)
        lettergrade += '-'; // 末尾是 0、1 或者 2 的成绩添加一个减号
```

要想发现这个错误可能非常困难，毕竟这段代码“看起来”是正确的。

177 为了避免此类问题，有些编码风格要求在 if 或 else 之后必须写上花括号（对 while 和 for 语句的循环体两端也有同样的要求）。这么做的好处是可以避免代码混乱不清，以后修改代码时如果想添加别的语句，也可以很容易地找到正确位置。



许多编辑器和开发环境都提供一种辅助工具，它可以自动地缩进代码以匹配其语法结构。善用此类工具益处多多。

悬垂 else

当一个 if 语句嵌套在另一个 if 语句内部时，很可能 if 分支会多于 else 分支。事实上，之前那个成绩转换的程序就有 4 个 if 分支，而只有 2 个 else 分支。这时候问题出现了：我们怎么知道某个给定的 else 是和哪个 if 匹配呢？

这个问题通常称作**悬垂 else**（dangling else），在那些既有 if 语句又有 if else 语句的编程语言中是个普遍存在的问题。不同语言解决该问题的思路也不同，就 C++ 而言，它规定 else 与离它最近的尚未匹配的 if 匹配，从而消除了程序的二义性。

当代码中 if 分支多于 else 分支时，程序员有时会感觉比较麻烦。举个例子来说明，对于添加加号减号的那个最内层的 if else 语句，我们用另外一组条件改写它：

```
// 错误：实际的执行过程并非像缩进格式显示的那样；else 分支匹配的是内层 if 语句
if (grade % 10 >= 3)
    if (grade % 10 > 7)
        lettergrade += '+'; // 末尾是 8 或者 9 的成绩添加一个加号
else
    lettergrade += '-'; // 末尾是 3、4、5、6 或者 7 的成绩添加一个减号！
```

从代码的缩进格式来看，程序的初衷应该是希望 else 和外层的 if 匹配，也就是说，我们希望当 grade 的末位小于 3 时执行 else 分支。然而，不管我们是什么意图，也不管程序如何缩进，这里的 else 分支其实是内层 if 语句的一部分。最终，上面的代码将在末位大于 3 小于等于 7 的成绩后面添加减号！它的执行过程实际上等价于如下形式：

```
// 缩进格式与执行过程相符，但不是程序员的意图
if (grade % 10 >= 3)
    if (grade % 10 > 7)
        lettergrade += '+'; // 末尾是 8 或者 9 的成绩添加一个加号
    else
        lettergrade += '-'; // 末尾是 3、4、5、6 或者 7 的成绩添加一个减号!
```

使用花括号控制执行路径

要想使 else 分支和外层的 if 语句匹配起来，可以在内层 if 语句的两端加上花括号，使其成为一个块：

```
//末尾是 8 或者 9 的成绩添加一个加号，末尾是 0、1 或者 2 的成绩添加一个减号
if (grade % 10 >= 3) {
    if (grade % 10 > 7)
        lettergrade += '+'; // 末尾是 8 或者 9 的成绩添加一个加号
} else // 花括号强迫 else 与外层 if 匹配
    lettergrade += '-'; // 末尾是 0、1 或者 2 的成绩添加一个减号
```

语句属于块，意味着语句一定在块的边界之内，因此内层 if 语句在关键字 else 前面的那个花括号处已经结束了。else 不会再作为内层 if 的一部分。此时，最近的尚未匹配的 if 是外层 if，也就是我们希望 else 匹配的那个。

178

5.3.1 节练习

练习 5.5：写一段自己的程序，使用 if else 语句实现把数字成绩转换成字母成绩的要求。

练习 5.6：改写上一题的程序，使用条件运算符（参见 4.7 节，第 134 页）代替 if else 语句。

练习 5.7：改正下列代码段中的错误。

```
(a) if (ival1 != ival2)
    ival1 = ival2
    else ival1 = ival2 = 0;

(b) if (ival < minval)
    minval = ival;
    occurs = 1;

(c) if (int ival = get_value())
    cout << "ival = " << ival << endl;
    if (!ival)
    cout << "ival = 0\n";

(d) if (ival = 0)
    ival = get_value();
```

练习 5.8：什么是“悬垂 else”？C++语言是如何处理 else 子句的？

5.3.2 switch 语句

switch 语句（switch statement）提供了一条便利的途径使得我们能够在若干固定选项中做出选择。举个例子，假如我们想统计五个元音字母在文本中出现的次数，程序逻辑应该如下所示：

- 从输入的内容中读取所有字符。
- 令每一个字符都与元音字母的集合比较。
- 如果字符与某个元音字母匹配，将该字母的数量加1。
- 显示结果。

例如，以（原书中）本章的文本作为输入内容，程序的输出结果将是：

```
Number of vowel a: 3195
Number of vowel e: 6230
Number of vowel i: 3102
Number of vowel o: 3289
Number of vowel u: 1033
```

179 要想实现这项功能，直接使用 switch 语句即可：

```
// 为每个元音字母初始化其计数值
unsigned aCnt = 0, eCnt = 0, iCnt = 0, oCnt = 0, uCnt = 0;
char ch;
while (cin >> ch) {
    // 如果 ch 是元音字母，将其对应的计数值加 1
    switch (ch) {
        case 'a':
            ++aCnt;
            break;
        case 'e':
            ++eCnt;
            break;
        case 'i':
            ++iCnt;
            break;
        case 'o':
            ++oCnt;
            break;
        case 'u':
            ++uCnt;
            break;
    }
}
// 输出结果
cout << "Number of vowel a: \t" << aCnt << '\n'
    << "Number of vowel e: \t" << eCnt << '\n'
    << "Number of vowel i: \t" << iCnt << '\n'
    << "Number of vowel o: \t" << oCnt << '\n'
    << "Number of vowel u: \t" << uCnt << endl;
```

switch 语句首先对括号里的表达式求值，该表达式紧跟在关键字 switch 的后面，可以是一个初始化的变量声明（参见 5.2 节，第 155 页）。表达式的值转换成整数类型，然后与每个 case 标签的值比较。

如果表达式和某个 case 标签的值匹配成功，程序从该标签之后的第一条语句开始执行，直到到达了 switch 的结尾或者是遇到一条 break 语句为止。

我们将在 5.5.1 节（第 170 页）详细介绍 break 语句，简言之，break 语句的作用是

中断当前的控制流。此例中, `break` 语句将控制权转移到 `switch` 语句外面。因为 `switch` 是 `while` 循环体内唯一的语句, 所以从 `switch` 语句中断出来以后, 程序的控制权将移到 `while` 语句的右花括号处。此时 `while` 语句内部没有其他语句要执行, 所以 `while` 会返回去再一次判断条件是否满足。

如果 `switch` 语句的表达式和所有 `case` 都没有匹配上, 将直接跳转到 `switch` 结构之后的第一条语句。刚刚说过, 在上面的例子中, 退出 `switch` 后控制权回到 `while` 语句的条件部分。

`case` 关键字和它对应的值一起被称为 **case 标签 (case label)**。case 标签必须是整型常量表达式 (参见 2.4.4 节, 第 58 页):

```
char ch = getVal();
int ival = 42;
switch(ch) {
case 3.14: // 错误: case 标签不是一个整数
case ival: // 错误: case 标签不是一个常量
//...
```

180

任何两个 `case` 标签的值不能相同, 否则就会引发错误。另外, `default` 也是一种特殊的 `case` 标签, 关于它的知识将在第 162 页介绍。

switch 内部的控制流

理解程序在 `case` 标签之间的执行流程非常重要。如果某个 `case` 标签匹配成功, 将从该标签开始往后顺序执行所有 `case` 分支, 除非程序显式地中断了这一过程, 否则直到 `switch` 的结尾处才会停下来。要想避免执行后续 `case` 分支的代码, 我们必须显式地告诉编译器终止执行过程。大多数情况下, 在下一个 `case` 标签之前应该有一条 `break` 语句。

然而, 也有一些时候默认的 `switch` 行为才是程序真正需要的。每个 `case` 标签只能对应一个值, 但是有时候我们希望两个或更多个值共享同一组操作。此时, 我们就故意省略掉 `break` 语句, 使得程序能够连续执行若干个 `case` 标签。

例如, 也许我们想统计的是所有元音字母出现的总次数:

```
unsigned vowelCnt = 0;
// ...
switch (ch)
{
    // 出现了 a、e、i、o 或 u 中的任意一个都会将 vowelCnt 的值加 1
    case 'a':
    case 'e':
    case 'i':
    case 'o':
    case 'u':
        ++vowelCnt;
        break;
}
```

在上面的代码中, 几个 `case` 标签连写在一起, 中间没有 `break` 语句。因此只要 `ch` 是元音字母, 不管到底是五个中的哪一个都执行相同的代码。

C++ 程序的形式比较自由, 所以 `case` 标签之后不一定非得换行。把几个 `case` 标签

写在一行里，强调这些 case 代表的是某个范围内的值：

```
switch (ch)
{
    // 另一种合法的书写形式
    case 'a': case 'e': case 'i': case 'o': case 'u':
        ++vowelCnt;
        break;
}
```

181

Best Practices

一般不要省略 case 分支最后的 break 语句。如果没写 break 语句，最好加一段注释说清楚程序的逻辑。

漏写 break 容易引发缺陷

有一种常见的错觉是程序只执行匹配成功的那个 case 分支的语句。例如，下面程序的统计结果是错误的：

```
// 警告：不正确的程序逻辑！
switch (ch) {
    case 'a':
        ++aCnt; // 此处应该有一条 break 语句
    case 'e':
        ++eCnt; // 此处应该有一条 break 语句
    case 'i':
        ++iCnt; // 此处应该有一条 break 语句
    case 'o':
        ++oCnt; // 此处应该有一条 break 语句
    case 'u':
        ++uCnt;
}
```

要想理解这段程序的执行过程，不妨假设 ch 的值是 'e'。此时，程序直接执行 case 'e' 标签后面的代码，该代码把 eCnt 的值加 1。接下来，程序将跨越 case 标签的边界，接着递增 iCnt、oCnt 和 uCnt。

Best Practices

尽管 switch 语句不是非得在最后一个标签后面写上 break，但是为了安全起见，最好这么做。因为这样的话，即使以后再增加新的 case 分支，也不用再在前面补充 break 语句了。

default 标签

如果没有任何一个 case 标签能匹配上 switch 表达式的值，程序将执行紧跟在 **default** 标签 (default label) 后面的语句。例如，可以增加一个计数值来统计非元音字母的数量，只要在 default 分支内不断递增名为 otherCnt 的变量就可以了：

```
// 如果 ch 是一个元音字母，将相应的计数值加 1
switch (ch) {
    case 'a': case 'e': case 'i': case 'o': case 'u':
        ++vowelCnt;
        break;
    default:
```

```

        ++otherCnt;
        break;
    }
}

```

在这个版本的程序中，如果 `ch` 不是元音字母，就从 `default` 标签开始执行并把 `otherCnt` 加 1。 ◀ 182

Best Practices

即使不准备在 `default` 标签下做任何工作，定义一个 `default` 标签也是有用的。其目的在于告诉程序的读者，我们已经考虑到了默认的情况，只是目前什么也没做。

标签不应该孤零零地出现，它后面必须跟上一条语句或者另外一个 `case` 标签。如果 `switch` 结构以一个空的 `default` 标签作为结束，则该 `default` 标签后面必须跟上一条空语句或一个空块。

switch 内部的变量定义

如前所述，`switch` 的执行流程有可能会跨过某些 `case` 标签。如果程序跳转到了某个特定的 `case`，则 `switch` 结构中该 `case` 标签之前的部分会被忽略掉。这种忽略掉一部分代码的行为引出了一个有趣的问题：如果被略过的代码中含有变量的定义该怎么办？

答案是：如果在某处一个带有初值的变量位于作用域之外，在另一处该变量位于作用域之内，则从前一处跳转到后一处的行为是非法行为。

```

case true:
    // 因为程序的执行流程可能绕开下面的初始化语句，所以该 switch 语句不合法
    string file_name;    // 错误：控制流绕过一个隐式初始化的变量
    int ival = 0;        // 错误：控制流绕过一个显式初始化的变量
    int jval;            // 正确：因为 jval 没有初始化
    break;
case false:
    // 正确：jval 虽然在作用域内，但是它没有被初始化
    jval = next_num();    // 正确：给 jval 赋一个值
    if (file_name.empty()) // file_name 在作用域内，但是没有被初始化
        // ...

```

假设上述代码合法，则一旦控制流直接跳到 `false` 分支，也就同时略过了变量 `file_name` 和 `ival` 的初始化过程。此时这两个变量位于作用域之内，跟在 `false` 之后的代码试图在尚未初始化的情况下使用它们，这显然是行不通的。因此 C++ 语言规定，不允许跨过变量的初始化语句直接跳转到该变量作用域内的另一个位置。

如果需要为某个 `case` 分支定义并初始化一个变量，我们应该把变量定义在块内，从而确保后面的所有 `case` 标签都在变量的作用域之外。

```

case true:
{
    // 正确：声明语句位于语句块内部
    string file_name = get_file_name();
    // ...
}
break;
case false:

```

```
if (file_name.empty()) // 错误: file_name 不在作用域之内
```

183

5.3.2 节练习

练习 5.9: 编写一段程序, 使用一系列 if 语句统计从 cin 读入的文本中有多少元音字母。

练习 5.10: 我们之前实现的统计元音字母的程序存在一个问题: 如果元音字母以大写形式出现, 不会被统计在内。编写一段程序, 既统计元音字母的小写形式, 也统计大写形式, 也就是说, 新程序遇到 'a' 和 'A' 都应该递增 aCnt 的值, 以此类推。

练习 5.11: 修改统计元音字母的程序, 使其也能统计空格、制表符和换行符的数量。

练习 5.12: 修改统计元音字母的程序, 使其能统计以下含有两个字符的字符序列的数量: ff、fl 和 fi。

练习 5.13: 下面显示的每个程序都含有一个常见的编程错误, 指出错误在哪里, 然后修改它们。

```
(a) unsigned aCnt = 0, eCnt = 0, iouCnt = 0;
```

```
    char ch = next_text();
```

```
    switch (ch) {
```

```
        case 'a': aCnt++;
```

```
        case 'e': eCnt++;
```

```
        default: iouCnt++;
```

```
    }
```

```
(b) unsigned index = some_value();
```

```
    switch (index) {
```

```
        case 1:
```

```
            int ix = get_value();
```

```
            ivec[ ix ] = index;
```

```
            break;
```

```
        default:
```

```
            ix = ivec.size()-1;
```

```
            ivec[ ix ] = index;
```

```
    }
```

```
(c) unsigned evenCnt = 0, oddCnt = 0;
```

```
    int digit = get_num() % 10;
```

```
    switch (digit) {
```

```
        case 1, 3, 5, 7, 9:
```

```
            oddcnt++;
```

```
            break;
```

```
        case 2, 4, 6, 8, 10:
```

```
            evencnt++;
```

```
            break;
```

```
    }
```

```
(d) unsigned ival=512, jval=1024, kval=4096;
```

```
    unsigned bufsize;
```

```
    unsigned swt = get_bufCnt();
```

```
    switch(swt) {
```

```
case ival:
    bufsize = ival * sizeof(int);
    break;
case jval:
    bufsize = jval * sizeof(int);
    break;
case kval:
    bufsize = kval * sizeof(int);
    break;
}
```

5.4 迭代语句

迭代语句通常称为循环，它重复执行操作直到满足某个条件才停下来。`while` 和 `for` 语句在执行循环体之前检查条件，`do while` 语句先执行循环体，然后再检查条件。

5.4.1 `while` 语句



只要条件为真，**`while`** 语句（`while statement`）就重复地执行循环体，它的语法形式是

```
while (condition)
    statement
```

在 `while` 结构中，只要 *condition* 的求值结果为真就一直执行 *statement*（常常是一个块）。*condition* 不能为空，如果 *condition* 第一次求值就得 `false`，*statement* 一次都不执行。

`while` 的条件部分可以是一个表达式或者是一个带初始化的变量声明（参见 5.2 节，第 155 页）。通常来说，应该由条件本身或者是循环体设法改变表达式的值，否则循环可能无法终止。



定义在 `while` 条件部分或者 `while` 循环体内的变量每次迭代都经历从创建到销毁的过程。

使用 `while` 循环

当不确定到底要迭代多少次时，使用 `while` 循环比较合适，比如读取输入的内容就是如此。还有一种情况也应该使用 `while` 循环，这就是我们想在循环结束后访问循环控制变量。例如：

```
vector<int> v;
int i;
// 重复读入数据，直至到达文件末尾或者遇到其他输入问题
while (cin >> i)
    v.push_back(i);
// 寻找第一个负值元素
auto beg = v.begin();
while (beg != v.end() && *beg >= 0)
    ++beg;
if (beg == v.end())
    // 此时我们知道 v 中的所有元素都大于等于 0
```